



KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Hafta 2 Laboratuvar Föyü

Fonksiyonlar ve Bellek Modeli

Doç. Dr. Tolga Berber · <https://avesis.ktu.edu.tr/tberber>🕒 **Tahmini süre:** 2 saat💻 **Gereksinimler:** C++20 derleyici (g++/clang++), bir metin editörü

Öğrenme fazları: 1 • Ön-öğretim 2 • İçerik 3 • Katılım 4 • Değerlendirme 5 • İzleme

1 • Ön-öğretim 📣 Neden Önemli?

Bir fonksiyona bir **Nokta** verdiniz, fonksiyon onu “öteledi”, ama **main**’e döndüğünüzde nokta hiç değişmemiş — ya da hiç beklemediğiniz halde değişmiş. Aynı biçimde, bir fonksiyon içinde **new** ile açtığınız bir belleği geri vermeyi unutmak, uzun çalışan bir programın belleğini sessizce şişirir. Bu iki şaşkınlığın da tek nedeni, parametreyi hangi biçimde geçtiğiniz ve belleğin kimin sorumluluğunda olduğudur. Bu föyde bu iki soruyu elinizle deneyerek yanıtlayacak; aşırı yüklenmiş fonksiyonlar, dört parametre geçme biçimi, **new/delete** ile bellek yönetimi ve kaynağı bir nesnenin ömrüne bağlayan RAII fikrini sıfırdan kuracaksınız.

1 • Ön-öğretim 🔄 Ön Bilgi

Bu föy YG I ders notu Bölüm 2 (Fonksiyonlar ve Bellek Modeli) üzerine kuruludur ve şu kavramları hazır varsayar: Hafta 1 — **namespace**, **<iostream>**, referans girişi (**int&**), **const**, **.hpp/.cpp** ayrımı ve CMake iskeleti; **geometri::Nokta** değer tipi. C’den: **malloc/free**, işaretçi ve fonksiyon tanımı/çağırısı (yalnız karşılaştırma noktası olarak anılır, yeniden öğretilmez). Bu lab Hafta 1’in **geometri::Nokta** yapısını hiç değiştirmeden aynen taşır.

Geçen hafta C++’ın temel sözdizimini ve zihniyetini gördünüz; referansları (**int&**) ve **.hpp/.cpp** + CMake iskeletini tanıdınız. Bu föyde o referans bilgisini derinleştirip fonksiyonların kendisini ve C++’ın bellek modelini işleyeceksiniz. Sırasıyla aynı adı taşıyan ama farklı davranan fonksiyonlar (aşırı yükleme) ve varsayılan parametreler yazacak, geçen haftadan taşıdığınız **geometri::Nokta** üzerinde dört farklı parametre geçme biçimini karşılaştıracak, yığın (*stack*) ile öbek (*heap*) arasındaki farkı **new/delete** ile elinizle deneyimleyecek, kasıtlı bir bellek sızıntısı deseninin neden tehlikeli olduğunu göreceksiniz. Son olarak, elle yönetimin risklerini azaltan bir fikre — RAII’ye — küçük bir **araclar::IntTampon** sarmalayıcısı yazarak ilk bakışı atacaksınız.

Bu föyü şöyle kullanın: her adımda önce *Ne yapacağız?* kutusundaki görevi tamamlayın (kodu yazın, ardından verilen kabuk komutunu çalıştırın), sonra kendi sonucunuzu *Beklenen* kutusu ve çıktı örneğiyle karşılaştırarak doğru yolda olduğunuzu hemen doğrulayın. Bir şey tutmazsa, sonraki adıma geçmeden o adımı düzeltin — her adım bir öncekinin üstüne inşa edilir. Her 3–4 adımda bir gelen *Kontrol Noktası*, o ana kadar yaptıklarınızı bir arada sınamanıza yarar. Bölümler ders notu Bölüm 2’nin dört ünitesiyle sırayla ilerler; her bölümün başında ilgili üniteye bir atıf bulacak, takıldığınızda oraya dönebileceksiniz.

1 Fonksiyonlar: Aşırı Yükleme ve Varsayılan Parametreler

(bkz. Ders notu Bölüm 2, Ünite 1 — Fonksiyonlar: Aşırı Yükleme ve Varsayılan Parametreler.)

Adım 1: Aynı adı taşıyan alan fonksiyonlarını aşırı yükleyin

❓ Ne yapacağız? `fonksiyon_ornekleri.cpp` adlı bir dosya oluşturacak, içine aynı ada (alan) sahip ama farklı parametre listeleriyle üç fonksiyon (kare, dikdörtgen, daire alanı) tanımlayacak, ardından bunları `main` içinden üç farklı çağrıyla derleyip çalıştıracaksınız.

❗ Neden? C’de her fonksiyonun benzersiz bir adı olmak zorundadır; kare, dikdörtgen ve daire alanı için ayrı ayrı `alanKare`, `alanDikdortgen`, `alanDaire` gibi adlar bulmanız gerekirdi. C++’ta ise aynı ada sahip birden çok fonksiyon tanımlayabilirsiniz, yeter ki parametre listeleri (sayı ve/veya tip bakımından) birbirinden farklı olsun — buna *aşırı yükleme* (*function overloading*) denir. Derleyici, siz `alan(4)` ya da `alan(3, 5)` yazdığınızda, çağrıdaki parametre listesine bakarak hangi sürümü çalıştıracağına derleme zamanında karar verir. Dikkat edilmesi gereken önemli bir nokta: bir fonksiyonu ayırt eden şey adı ve parametre listesidir (buna *imza* denir), **dönüş tipi değildir** — yalnızca dönüş tipiyle birbirinden ayrılan iki fonksiyon tanımlamaya çalışırsanız derleyici hata verir, çünkü çağrı anında hangisinin kastedildiğine karar verecek bir bilgi (parametre listesi) yoktur.

```
5 int alan(int kenar) {
6     return kenar * kenar;
7 }
8
9 int alan(int taban, int yukseklik) {
10    return taban * yukseklik;
11 }
12
13 double alan(double yaricap) {
14    return 3.14159 * yaricap * yaricap;
15 }
```

Analoji

Aşırı yükleme, gündelik dilde “aç” fiilinin farklı bağlamlarda kullanılmasına benzer: “kapıyı aç”, “dosyayı aç”, “mağazayı aç” — aynı fiili (aynı adı) söylersiniz, ama sizi dinleyen kişi (derleyici) hangi “parametreyle” (kapı mı, dosya mı, mağaza mı) konuştuğunuza bakarak hangi eylemi kastettiğinizi otomatik çıkarır. `alan(4)` ile `alan(3, 5)` de aynı şekilde, verdiğiniz parametre sayısına göre derleyici tarafından ayrıştırılır.

Üç fonksiyonu tanımladıktan sonra, dosyanın sonuna aşağıdaki gibi kısa bir `main` ekleyin (yalnızca alan çağrılarıyla):

```
1 int main() {
2     std::cout << "kare alani (kenar=4): " << alan(4) << '\n';
3     std::cout << "dikdortgen alani (3x5): " << alan(3, 5) << '\n';
4     std::cout << "daire alani (r=2.0): " << alan(2.0) << '\n';
5     return 0;
6 }
```

Terminal

```
$ g++ -std=c++20 fonksiyon_ornekleri.cpp -o fonksiyon_demo
$ ./fonksiyon_demo
```

Adım 1 (devamı)

👁 **Beklenen sonuç:** Derleme hatasız tamamlanır; program üç satırda üç farklı alan çağrısının sonucunu yazdırır. Aynı ada rağmen her çağrının doğru sürüme (kare/dikdörtgen/daire) gittiğini görürsünüz.

Beklenen Çıktı

```
karealani (kenar=4): 16
dikdortgenalani (3x5): 15
dairealani (r=2.0): 12.5664
```

Adım 2: guc fonksiyonuna varsayılan parametre ekleyin

❓ **Ne yapacağız?** Aynı dosyaya, `üs` parametresi verilmezse otomatik olarak 2 kabul edilen bir `guc(int taban, int us = 2)` fonksiyonu ekleyecek, `main` içinde bu fonksiyonu hem varsayılan değerle hem de üssü kendiniz belirterek iki kez çağıracaksınız.

📌 **Neden?** *Varsayılan parametre (default parameter)*, bir fonksiyon bildiriminde bir parametreye öntanımlı bir değer vermenizi sağlar; çağırın taraf o parametreyi hiç yazmazsa derleyici otomatik olarak bu değeri kullanır. `guc(3)` çağrısı böylece `guc(3, 2)` ile aynı anlama gelir, ama siz gerektiğinde `guc(3, 3)` yazarak varsayılanı geçersiz kılabilirsiniz. Kural şudur: varsayılan değeri olan parametreler parametre listesinin sonunda kümelenmelidir — `taban`'dan önce bir varsayılan parametre koyup `us`'u varsayılanı bırakamazsınız, çünkü derleyici çağrıdaki değerleri soldan sağa eşleştirir ve hangi parametrenin atlandığını yalnızca sondan kesin biçimde çıkarabilir.

```
19 long guc(int taban, int us = 2) {
20     long sonuc = 1;
21     for (int i = 0; i < us; ++i) {
22         sonuc *= taban;
23     }
24     return sonuc;
25 }
```

Analoji

Varsayılan parametre, bir kahve dükkanında "bir kahve, lütfen" demenize benzer: özel bir boy belirtmezseniz size otomatik olarak orta boy gelir; isterseniz "büyük kahve" diyerek bu varsayılanı geçersiz kılarırsınız. `guc(3)` de aynı şekilde "üssü belirtmedim, 2 kabul et" demektir.

`main` fonksiyonunuzu, önceki adımdaki üç alan çağrısının altına aşağıdaki iki satırı da ekleyecek biçimde güncelleyin:

```
1 std::cout << "guc(3) [varsayılan us=2]: " << guc(3) << '\n';
2 std::cout << "guc(3, 3) [us=3]: " << guc(3, 3) << '\n';
```

Terminal

```
$ g++ -std=c++20 fonksiyon_ornekleri.cpp -o fonksiyon_demo
$ ./fonksiyon_demo
```

👁 **Beklenen sonuç:** Önceki üç satırın altında artık iki satır daha görürsünüz: `guc(3)` varsayılan üsle (2) 9 sonucunu, `guc(3, 3)` ise belirtilen üsle (3) 27 sonucunu verir.

Adım 2 (devamı) Beklenen Çıktı

```
kare alani (kenar=4): 16
dikdortgen alani (3x5): 15
daire alani (r=2.0): 12.5664
guc(3) [varsayılan us=2]: 9
guc(3, 3) [us=3]: 27
```

 Dikkat

Aşırı yükleme ile varsayılan parametreyi bir arada kullanırken dikkatli olun: eğer `guc(int taban, int us = 2)` yanında bir de `guc(int taban)` fonksiyonu tanımlarsanız, `guc(3)` çağrısının hangi sürümü kastettiği derleyici için belirsizleşir (*ambiguity*) ve derleme hatası alırsınız. Bu haftada bu durumla karşılaşmayacaksınız, ama ileride aşırı yükleme ile varsayılan parametreyi birlikte tasarlarken bu riski aklınızda tutun.

✓ Kontrol Noktası — devam etmeden doğrulayın

`fonksiyon_ornekleri.cpp` dosyanız artık üç aşırı yüklenmiş alan fonksiyonu ve varsayılan parametrelili bir `guc` fonksiyonu içeriyor; `./fonksiyon_demo` çalıştırdığınızda beş satırlık çıktının tamamını gördüğünüzden emin olun.

2 Parametre Geçme: Değer, Referans, const Referans, İşaretçi

(bkz. *Ders notu Bölüm 2, Ünite 2 — Parametre Geçme: Değer, Referans, const Referans, İşaretçi.*)

Adım 3: geometri::Nokta yapısını Hafta 1'den taşıyın

? Ne yapacağız? Geçen haftaki `nokta.hpp` ve `nokta.cpp` dosyalarını, içerikleri birebir aynı kalacak biçimde bu haftaki çalışma klasörünüze kopyalayacak (ya da elinizde yoksa aynı içerikle yeniden oluşturacak) ve tek başına derlendiğini doğrulayacaksınız.

i Neden? Bu hafta yeni bir veri tipiyle uğraşmak yerine, geçen hafta kurduğunuz `geometri::Nokta` yapısını aynen kullanacaksınız — amaç, parametre geçme biçimlerinin veri tipinden değil, fonksiyon imzasından kaynaklanan bir fark olduğunu net biçimde görmektir. Aynı `Nokta` üzerinde dört farklı fonksiyon çağırarak, hangisinin çağırının nesnesini değiştirdiğini karşılaştıracaksınız.

```
8 struct Nokta {
9     double x;
10    double y;
11 };

15 Nokta topla(const Nokta& a, const Nokta& b);
16 void yazdir(const Nokta& n);

9 Nokta topla(const Nokta& a, const Nokta& b) {
10     return Nokta{a.x + b.x, a.y + b.y};
11 }

15 void yazdir(const Nokta& n) {
16     std::cout << "(" << n.x << ", " << n.y << ")";
17 }
```

Adım 3 (devamı)**>_ Terminal**`$ g++ -std=c++20 -c nokta.cpp -o nokta.o`

🕒 Beklenen sonuç: Komut hatasız tamamlanır ve dizinde küçük bir nokta.o nesne dosyası oluşur (henüz main içermediği için tek başına çalıştırılabilir değildir).

Adım 4: Dört farklı parametre geçme fonksiyonunu yazın

❓ Ne yapacağız? parametre_gecme_ornekleri.cpp dosyasında, aynı Nokta üzerinde çalışan dört fonksiyon yazacaksınız: değere göre öteleyen oteleDeger, referansla öteleyen oteleReferans, yalnızca okuyan oteleConstReferans ve işaretçiyle öteleyen oteleIsaretci.

📌 Neden? Bir parametreyi fonksiyona dört farklı biçimde geçirebilirsiniz, her birinin çağırana etkisi ve maliyeti farklıdır: **değere göre geçme** (Nokta n) çağrı anında nesnenin bir kopyasını oluşturur — fonksiyon içindeki değişiklik yalnızca bu kopyayı etkiler, çağıranın orijinali dokunulmamış kalır. **Referansla geçme** (Nokta& n) ise Hafta 1’de gördüğünüz referans fikrinin devamıdır: n, çağıranın nesnesine ikinci bir isimdir, kopya oluşmaz, fonksiyon içindeki her değişiklik doğrudan çağırana yansır. **const referansla geçme** (const Nokta& n) bu ikisinin en faydalı bileşimidir: kopya maliyeti yoktur (referans gibi) ama derleyici n üzerinde değişiklik yapmanıza izin vermez — büyük bir nesneyi yalnızca okumak istediğinizde varsayılan tercihiniz bu olmalıdır. **İşaretçiyle geçme** (Nokta* n) C’den tanıdık geldiğiniz biçimdir; farkı, işaretçinin nullptr yani “hiçbir nesneyi göstermiyor” durumunda olabilmesidir — bu yüzden işaretçi üzerinden erişmeden önce her zaman nullptr kontrolü yapmanız gerekir. İşaretçi, referansın karşılayamadığı tek durumda tercih edilir: parametrenin isteğe bağlı olduğu, yani hiç gönderilmeyebileceği durumlarda.

```

8 void oteleDeger(Nokta n) {
9     n.x += 10.0;
10    n.y += 10.0;
11 }

15 void oteleReferans(Nokta& n) {
16     n.x += 10.0;
17     n.y += 10.0;
18 }

22 double oteleConstReferans(const Nokta& n) {
23     // yalnızca okur; n üzerinde değişiklik yapamaz (const)
24     return n.x + n.y;
25 }

29 void oteleIsaretci(Nokta* n) {
30     if (n == nullptr) {
31         return;
32     }
33     n->x += 10.0;
34     n->y += 10.0;
35 }

```

Adım 4 (devamı)

Analoji

Değere göre geçme, bir belgenin fotokopisini karşı tarafa vermeye benzer: karşı taraf fotokopi üzerinde ne yazarsa yazsın, elinizdeki orijinal belge değişmeden kalır. Referansla geçme ise belgenin kendisini elden vermeye benzer: karşı taraf belge üzerinde yaptığı her değişikliği kalıcı kılar, çünkü ortada tek bir belge vardır (Hafta 1'in "lakap" benzetmesinin bir parametre geçme uzantısıdır). İşaretçiyle geçme ise belgenin bulunduğu dolabın numarasını vermeye benzer: numara boş olabilir (hiçbir dolabı göstermeyebilir — `nullptr`), numarayı alan kişi önce "dolap gerçekten dolu mu" diye kontrol etmek zorundadır; aksi hâlde boş bir dolabı açmaya çalışır.

Karar tablosu düzeyinde bir özet aklınızda kalsın:

- Küçük tip (`int`, `double` gibi) → değere göre geçirin.
- Büyük tip, fonksiyon içinde geçilecek → `T&` kullanın.
- Büyük tip, fonksiyon yalnızca okuyacak → `const T&` kullanın (varsayılan tercihiniz bu olsun).
- Parametre isteğe bağlı/boş olabilir → `T*` kullanın (`nullptr` kontrolüyle birlikte).

Ortak Hata / Tuzak

`oteleIsaretci` içinde `if (n == nullptr) return;` satırını atlamak, işaretçi gerçekten boş geldiğinde (örneğin çağıran `oteleIsaretci(nullptr);` yazarsa) `n->x` satırında tanımsız davranışa (*undefined behavior*) yol açar. İşaretçi parametre alan her fonksiyonda, gövdenin ilk satırı alışkanlık olarak bir `nullptr` kontrolü olmalıdır.

Beklenen sonuç: Dört fonksiyon da hatasız derlenir; henüz `main` yazmadığınız için çalıştırılabilir bir çıktı almazsınız — bu bir sonraki adımda gelecek.

Adım 5: `main` içinde dört biçimi karşılaştırıp gözlemleyin

Ne yapacağız? Aynı başlangıç Nokta'sından türetilen dört ayrı kopya üzerinde sırasıyla dört öteleme fonksiyonunu çağırarak, sonuçları `geometri::yazdir` ile ekrana basarak hangi çağırandaki noktanın gerçekten değiştiğini karşılaştıracaksınız.

Neden? Aynı Nokta{1.0, 1.0} başlangıç değerini dört kez kopyalayıp her birini farklı bir fonksiyona geçirdiğinizde, yalnızca referans ve işaretçi sürümlerinin çağırandaki nesneyi kalıcı olarak değiştirdiğini; değere göre ve `const` referansla geçirilen sürümlerin çağırandaki nesneyi hiç değiştirmediklerini doğrudan gözlemleyeceksiniz.

```
39 int main() {
40     Nokta baslangic{1.0, 1.0};
41
42     Nokta degerNoktasi = baslangic;
43     oteleDeger(degerNoktasi);
44     std::cout << "deger ile -> otelemeden sonra (degismedi): ";
45     geometri::yazdir(degerNoktasi);
46     std::cout << '\n';
47
48     Nokta referansNoktasi = baslangic;
49     oteleReferans(referansNoktasi);
```

Adım 5 (devamı)

```

50     std::cout << "referans ile -> otelemeden sonra (degisti) : ";
51     geometri::yazdir(referansNoktasi);
52     std::cout << '\n';
53
54     Nokta constNoktasi = baslangic;
55     double toplam = oteleConstReferans(constNoktasi);
56     std::cout << "const ref ile -> nokta degismedi, okundu : ";
57     geometri::yazdir(constNoktasi);
58     std::cout << " (x+y=" << toplam << ")\n";
59
60     Nokta isaretciNoktasi = baslangic;
61     oteleIsaretci(&isaretciNoktasi);
62     std::cout << "isaretci ile -> otelemeden sonra (degisti) : ";
63     geometri::yazdir(isaretciNoktasi);
64     std::cout << '\n';
65
66     return 0;
67 }

```

> Terminal

```

$ g++ -std=c++20 parametre_gecme_ornekleri.cpp nokta.cpp -o parametre_demo
$ ./parametre_demo

```

👁 Beklenen sonuç: Dört satırlık çıktıda, "değer ile" ve "const ref ile" satırlarındaki noktanın başlangıç değeri (1, 1) olarak kaldığını, "referans ile" ve "işaretçi ile" satırlarındaki noktanın ise (11, 11)'e ötelendiğini görürsünüz.

🖥 Beklenen Çıktı

```

deger ile      -> otelemeden sonra (degismedi): (1, 1)
referans ile   -> otelemeden sonra (degisti)  : (11, 11)
const ref ile  -> nokta degismedi, okundu     : (1, 1) (x+y=2)
isaretci ile   -> otelemeden sonra (degisti)  : (11, 11)

```

✅ Kontrol Noktası — devam etmeden doğrulayın

Dört satırlık çıktıda hangi iki sürümün noktayı gerçekten değiştirdiğini (referans ve işaretçi), hangi iki sürümün değiştirmediğini (değer ve const referans) sözlü olarak kendinize açıklayabildiğinizden emin olun — bu ayırım Ünite 3'ün temelini oluşturuyor.

3 Bellek Modeli: Yığın, Öbek, new/delete, nullptr

(bkz. Ders notu Bölüm 2, Ünite 3 — Bellek Modeli: Yığın, Öbek, new/delete, nullptr.)

Adım 6: Yığındaki ve öbekteki nesneleri karşılaştırıp nullptr kontrolü ekleyin

❓ Ne yapacağız? bellek_modeli_ornekleri.cpp dosyasında, yığında (*stack*) otomatik ömürlü yerel bir Nokta oluşturan bir fonksiyon, new ile öbekte (*heap*) tekil bir Nokta ayırıp delete ile serbest bırakan bir fonksiyon ve bir işaretçinin nullptr olup olmadığını kontrol eden bir fonksiyon yazacak, main içinde üçünü de çağıracağız.

Adım 6 (devamı)

i Neden? Yığın, bir fonksiyon/blok kapsamına girdiğinizde otomatik ayrılan, kapsam sona erdiğinde otomatik geri alınan bir bellek alanıdır — boyutu ve ömrü sizin elinizde değildir, derleyici yönetir. Öbek ise tam tersine, siz `new` ile açıkça istemedikçe size ait olmayan, siz `delete` çağırana kadar (kapsamdan tamamen bağımsız biçimde) yaşamaya devam eden bir bellek alanıdır. `new Nokta{7.0, 7.0}` yazdığınızda C'nin `malloc`'undan farklı olarak hem doğru boyutta bellek ayrılır hem de nesnenin ilk değerleri (kurucu adı verilen, Hafta 3'te biçimsel olarak tanımlayacağınız özel bir mekanizma aracılığıyla) atanır — `malloc` yalnız ham bellek verir, içeri sıfırlamaz veya başlatmaz. `nullptr`, C'nin `NULL/0` makrosuna kıyasla tip güvenli bir boş işaretçi değeridir; bir işaretçiyi kullanmadan (dereference etmeden) önce `nullptr` olup olmadığını kontrol etmek, C++'ta edinmeniz gereken temel bir alışkanlıktır.

```

8 void stackOrnegi() {
9     Nokta yerel{5.0, 5.0};
10    std::cout << "stack nesnesi adresi: " << &yerel << '\n';
11 }

15 void newDeleteOrnegi() {
16     Nokta* dinamik = new Nokta{7.0, 7.0};
17     std::cout << "dinamik nesne adresi: " << dinamik << '\n';
18     delete dinamik;
19     dinamik = nullptr;
20 }

24 void nullptrKontrolOrnegi() {
25     Nokta* p = nullptr;
26     if (p == nullptr) {
27         std::cout << "isaretci bos (nullptr), erisim yapilmiyor\n";
28     } else {
29         std::cout << "isaretci dolu: " << p->x << '\n';
30     }
31 }

```

Analoji

Yığını masaüstünüze, öbeği kiraladığınız bir depoya benzetebilirsiniz: masaüstündeki kağıtlar (yığındaki değişkenler) işiniz bitince kendiliğinden toplanır (kapsam sona erer ermez otomatik geri alınır); depo ise siz gidip kiralamadıkça (`new`) size ait olmaz, siz anahtarı iade etmedikçe (`delete`) de boşalmaz.

Üç fonksiyonu tanımladıktan sonra dosyanın sonuna aşağıdaki `main`'i ekleyin:

```

1 int main() {
2     stackOrnegi();
3     newDeleteOrnegi();
4     nullptrKontrolOrnegi();
5     return 0;
6 }

```

> Terminal

```

$ g++ -std=c++20 bellek_modeli_ornekleri.cpp nokta.cpp -o bellek_demo
$ ./bellek_demo

```

Adım 6 (devamı)

🎯 **Beklenen sonuç:** Program üç satır yazdırır: ilk ikisi bir yığın adresi ve bir öbek adresi (sizin bilgisayarınızda aşağıdakinden farklı onaltılık değerler görürsünüz, önemli olan ikisinin de birer geçerli adres olması ve birbirinden farklı bellek bölgelerinde durması), üçüncüsü nullptr kontrolünün doğru dallandığını gösteren bir mesajdır.

Beklenen Çıktı

```
stack nesnesi adresi: 0x7ffee3a1b840
dinamik nesne adresi: 0x600002a04030
isaretci bos (nullptr), erisim yapilmiyor
```

Adım 7: Kasıtlı bir bellek sızıntısı ve sarkan işaretçi deseni ekleyin

❓ **Ne yapacağız?** Aynı dosyaya, KASITLI olarak hatalı iki fonksiyon ekleyeceksiniz: sızıntı0rneği bir bellek sızıntısı deseni, sarkan0rneği bir sarkan işaretçi (*dangling pointer*) deseni gösterecek. Bu iki fonksiyonu main içinden **çağırılmayacaksınız** — yalnızca derlenebilir birer örnek desen olarak dosyada duracaklar.

📌 **Neden?** Bellek sızıntısı (*memory leak*), new ile ayrılan bir belleğin karşılığı olan delete hiç çağrılmadan o belleğe erişiminizi kaybetmenizle oluşur — işaretçi kapsam dışına çıksa bile, ayrılan bellek program sonlanana kadar geri alınamaz. sızıntı0rneği içinde p önce ilk Nokta'yı gösterir, sonra aynı p ikinci bir new çağrısının sonucuna yeniden atanır — ilk adres artık hiçbir değişkende tutulmadığı için o belleğe bir daha asla delete çağırılmazsınız. Sarkan işaretçi ise delete edilmiş belleği hâlâ gösteren bir işaretçinin kullanılmasıdır; sarkan0rneği içinde p, delete p; satırından sonra artık geçersiz bir adresi gösterir — bu haldeki bir işaretçi üzerinden p->x gibi bir erişim yapmak tanımsız davranıştır (programınız çökebilir, yanlış bir değer okuyabilir ya da tesadüfen "çalışıyormuş" gibi görünebilir, hiçbiri güvenilir değildir). Bu yüzden her iki fonksiyon da kaynak kodda yorumla açıkça işaretlenmiş, gerçek bir erişim (dereference) tetiklemeyecek biçimde yazılmıştır — amaç desenin kendisini görmenizdir, programınızı gerçekten bozmak değil.

```
35 // DIKKAT: bu fonksiyon KASITLI olarak hatalıdır; main() tarafından CAGRILMAZ.
36 // new ile ayrılan bellegin karsiligi hicbir zaman delete edilmiyor.
37 void sızıntı0rneği() {
38     Nokta* p = new Nokta{1.0, 1.0};
39     p = new Nokta{2.0, 2.0}; // ilk ayrılan adres burada kaybolur, artık delete
        edilemez
40     // delete p; // <- kasitli olarak cagrilmadi: bellek sızıntısı bu satirin
        eksikliginden dogar
41     (void)p;
42 }

46 // DIKKAT: bu fonksiyon KASITLI olarak sarkan isaretci DESENINI gosterir;
47 // main() tarafından CAGRILMAZ ve p üzerinden gercek bir erisim (dereference)
        YAPILMAZ.
48 void sarkan0rneği() {
49     Nokta* p = new Nokta{3.0, 3.0};
50     delete p;
51     // p artık sarkan (dangling) bir isaretci: delete edilmiş bellegi gosteriyor.
52     // İyi aliskanlik: delete sonrası "p = nullptr;" yapmaktır (burada bilerek atlandı
        ).
53     // p->x = 9.0; // <- YAPILMAMALIDIR: tanımsız davranis (UB); bu satir hicbir zaman
        calismaz.
54 }
```

Adım 7 (devamı)**Ortak Hata / Tuzak**

Bu iki fonksiyon, öğrencilerin gerçek projelerde en sık düştüğü iki hatanın küçültülmüş örnekleridir: (1) bir işaretçiyi yeni bir new sonucuna atamadan önce eskisini delete etmeyi unutmak (sızıntı), (2) delete çağırdıktan sonra işaretçiyi nullptr'a çekmeyi atlayıp o işaretçiyi başka bir yerde yanlışlıkla tekrar kullanmak (sarkan işaretçi). İyi alışkanlık: her delete p; satırından hemen sonra p = nullptr; yazmaktır — tıpkı bu föyün bir önceki adımındaki newDelete0rneği fonksiyonunun yaptığı gibi. Böylece p kazara tekrar kullanılsa bile, önceki adımda yazdığınız nullptr kontrolü sizi tanımsız davranıştan korur.

>_ Terminal

```
$ g++ -std=c++20 bellek_modeli_ornekleri.cpp nokta.cpp -o bellek_demo
$ ./bellek_demo
```

👁 Beklenen sonuç: Dosya, iki yeni fonksiyon eklenmiş olmasına rağmen hatasız derlenir (yalnızca birer desen olarak durdukları için); main içeriği değişmediğinden çalıştırdığınızda önceki adımla birebir aynı üç satırlık çıktıyı görürsünüz.

🖥 Beklenen Çıktı

```
stack nesnesi adresi: 0x7ffee3a1b840
dinamik nesne adresi: 0x600002a04030
isaretci bos (nullptr), erisim yapilmiyor
```

✅ Kontrol Noktası — devam etmeden doğrulayın

sizinti0rneği ve sarkan0rneği fonksiyonlarınızın main içinden çağrılmadığından emin olun; bu föyde hiçbir tanımsız davranışı gerçekten tetiklemeyeceksiniz, yalnızca deseni tanıyacaksınız.

4 RAII'ye İlk Bakış: Kaynağı Nesne Ömrüne Bağlamak

(bkz. Ders notu Bölüm 2, Ünite 4 — RAII'ye İlk Bakış: Kaynağı Nesne Ömrüne Bağlamak.)

Adım 8: IntTampon yapısını bildirin (int_tampon.hpp)

❓ Ne yapacağız? int_tampon.hpp adlı bir dosyada, araçlar ad alanı içinde IntTampon adlı, bir işaretçi (veri) ve bir tamsayı (boyut) alanı taşıyan; ayrıca bir kurucu (IntTampon(int n)) ve bir yıkıcı (~IntTampon()) bildiren bir yapı yazacaksınız.

📌 Neden? Bu haftaya kadar elle new/delete yönetiminin riskini gördünüz: her new için karşılığı bir delete gerekir; erken bir return, atlanan bir kod yolu ya da basitçe unutmak tek başına sızıntıya yeter. RAII (*Resource Acquisition Is Initialization*) fikri, bir kaynağın (burada: dinamik bir tamsayı dizisi) edinimini bir nesnenin **kuruluşuna**, o kaynağın bırakılmasını ise aynı nesnenin **kapsam sonunda otomatik çalışan özel bir fonksiyonuna** bağlar. Bu haftada “kurucu” ve “yıkıcı” terimlerini biçimsel olarak tanımlamayacaksınız (bu, Hafta 3'ün konusudur) — yalnızca kapsam sona erdiğinde otomatik çalışan bir temizlik mekanizmasının var olduğunu gözlemleyeceksiniz.

Adım 8 (devamı)

```

8   int* veri;
9   int boyut;

12  IntTampon(int n);
13  ~IntTampon();

```

👁 **Beklenen sonuç:** `int_tampon.hpp` dosyanız `#pragma once` ile başlar, araçlar ad alanı içinde `IntTampon` yapısının alanlarını ve iki özel fonksiyon bildirimini içerir; dosya henüz tek başına derlenip çalıştırılabilir bir program oluşturmaz.

Adım 9: Kurucu ve yıkıcı gövdelerini tanımlayın (`int_tampon.cpp`)

❓ **Ne yapacağız?** `int_tampon.cpp` dosyasında, kurucunun `new int[n]` ile belleği ayırıp ekrana "tampon oluşturuldu" yazan gövdesini, yıkıcının ise `delete[] veri` ile belleği serbest bırakıp ekrana "tampon temizlendi" yazan gövdesini tanımlayacaksınız.

📌 **Neden?** Kurucu gövdesi, `IntTampon` nesnesi oluşturulduğu anda otomatik çalışır: `veri`'yi `n` elemanlık bir diziye işaret edecek biçimde ayarlar. Yıkıcı gövdesi ise nesne kapsam dışına çıktığı anda — siz hiçbir şey yazmasanız bile — otomatik çalışır ve `delete[]` ile ayrılan belleği serbest bırakır. Konsola bastığınız "oluşturuldu"/"temizlendi" mesajları, bu iki anın gerçekten ne zaman tetiklendiğini gözünüzle görmeyi sağlayacak.

```

8  IntTampon::IntTampon(int n) {
9      veri = new int[n];
10     boyut = n;
11     std::cout << "tampon olusturuldu (boyut=" << boyut << ")\n";
12 }

16 IntTampon::~IntTampon() {
17     delete[] veri;
18     std::cout << "tampon temizlendi\n";
19 }

```

>_ Terminal

```
$ g++ -std=c++20 -c int_tampon.cpp -o int_tampon.o
```

👁 **Beklenen sonuç:** Komut hatasız tamamlanır ve dizinde bir `int_tampon.o` nesne dosyası oluşur (henüz `main` içermediği için tek başına çalıştırılabilir değildir).

Adım 10: Elle yönetilen bir tamsayı dizisini `new[]/delete[]` ile ayırıp serbest bırakın

❓ **Ne yapacağız?** `int_tampon_demo.cpp` dosyasında, `main` içinde `new int[5]` ile beş elemanlık bir tamsayı dizisi ayırarak, elemanlarını bir döngüyle dolduracak, ardından `delete[]` ile serbest bırakacaksınız — henüz `IntTampon` kullanmadan, tamamen elle.

📌 **Neden?** Bir önceki üniteden hatırlayacağınız gibi, `new[]` ile ayrılan bir dizinin serbest bırakılması tamamen sizin elinizdedir: `delete[]` satırını unutursanız (ya da bir hata nedeniyle o satıra hiç ulaşılmazsa) bu dizi sızıntıya dönüşür. Bu adımda o riski bilerek elle yöneteceksiniz; bir sonraki adımda aynı işi yapan ama kendini otomatik temizleyen `IntTampon` sürümünü yan yana koyacaksınız.

```

7   std::cout << "-- elle yonetim --\n";
8   int* elleTampon = new int[5];

```

Adım 10 (devamı)

```

9   for (int i = 0; i < 5; ++i) {
10      elleTampon[i] = i * i;
11   }
12   std::cout << "elle tampon dolduruldu\n";
13   // Not: burada delete[] cagrısı UNUTULURSA bellek sızıntısı olur.
14   delete[] elleTampon;
15   std::cout << "elle tampon serbest bırakıldı\n";

```

>_ Terminal

```

$ g++ -std=c++20 int_tampon_demo.cpp int_tampon.cpp -o int_tampon_demo
$ ./int_tampon_demo

```

👁 **Beklenen sonuç:** Program üç satır yazdırır: elle yönetimin başladığını, dizinin dolduğunu ve delete[] ile serbest bırakıldığını gösteren mesajlar. IntTampon henüz hiç kullanılmadığı için "tampon oluşturuldu"/"tampon temizlendi" mesajlarını bu adımda görmezsiniz.

🖥 Beklenen Çıktı

```

-- elle yönetim --
elle tampon dolduruldu
elle tampon serbest bırakıldı

```

Adım 11: RAII bloğunu ekleyip temizlenme sırasını gözlemleyin

❓ **Ne yapacağız?** Aynı main fonksiyonuna, { } ile sınırlanmış küçük bir blok içinde bir araclar::IntTampon nesnesi oluşturup dolduracak, blok kapandıktan sonra da bir satır daha yazdıracaksınız.

📌 **Neden?** Bu adımda, elle yönetimle aynı işi yapan ama kaynağını bir nesnenin ömrüne bağlayan RAII sürümünü çalışırken göreceksiniz: tampon nesnesi bloğun başında oluşturulduğunda kurucu otomatik çalışıp belleği ayırır ve "tampon oluşturuldu" yazar; blok kapanıp tampon kapsam dışına çıktığında ise — siz hiçbir delete[] satırı yazmadan — yıkıcı otomatik çalışıp belleği serbest bırakır ve "tampon temizlendi" yazar. Elle yönetimden farkı budur: temizliği hatırlamanız gerekmez, derleyici sizin için garanti eder.

```

19   std::cout << "-- RAII demo --\n";
20   {
21       araclar::IntTampon tampon(5);
22       for (int i = 0; i < tampon.boyut; ++i) {
23           tampon.veri[i] = i * i;
24       }
25       std::cout << "tampon kullaniliyor\n";
26   } // blok sonu: yıkıcı otomatik çalışır, tampon kendini temizler
27   std::cout << "blok dışına cikildi\n";

```

Adım 11 (devamı)

Analoji

IntTampon, bir otel oda kartına benzer: kartı check-in’de alırsınız — oda (kaynak) o andan itibaren sizindir (nesnenin kuruluşu, kaynağın edinimidir). Check-out’ta kartı elden vermeseniz bile otel, süreniz dolduğunda kapıyı kendiliğinden kilitleyip odayı boşaltır (kapsam sonu, otomatik bırakmadır) — anahtarı elle “iade etmeyi hatırlamanıza” gerek kalmaz. Elle yönettiğiniz `new[]/ delete[]` dizisi ise check-out’ta anahtarı elden teslim etmeniz gereken eski usul bir kiralamaya benzer: unutursanız, oda sonsuza kadar “kayıp” kalır.

Terminal

```
$ g++ -std=c++20 int_tampon_demo.cpp int_tampon.cpp -o int_tampon_demo
$ ./int_tampon_demo
```

Beklenen sonuç: Önceki üç satırın altında dört satır daha görürsünüz. Kritik olan sıradır: “tampon oluşturuldu” mesajı bloğa girer girmez, “tampon kullanılıyor” bloğun içinde, “tampon temizlendi” mesajı ise **blok kapanır kapanmaz, “blok disina cikildi” satırından önce** görünür — yıkıcının gerçekten kapsam sonunda, siz hiçbir şey yazmadan otomatik çalıştığının kanıtıdır.

Beklenen Çıktı

```
-- elle yonetim --
elle tampon dolduruldu
elle tampon serbest birakildi
-- RAII demo --
tampon olusturuldu (boyut=5)
tampon kullaniliyor
tampon temizlendi
blok disina cikildi
```

✓ Kontrol Noktası — devam etmeden doğrulayın

Çıktınızda “tampon temizlendi” satırının “blok disina cikildi” satırından **önce** geldiğini doğrulayın — sıra ters görünüyorsa IntTampon nesnenizi yanlış kapsamda (bloğun dışında) oluşturmuş olabilirsiniz.

Adım 12: CMakeLists.txt oluşturup tüm hedefleri tek seferde derleyin

? Ne yapacağız? Şu ana kadar dört ayrı programı (fonksiyon_demo, parametre_demo, bellek_demo, int_tampon_demo) her seferinde elle g++ komutu yazarak derlediniz. Şimdi tek bir CMakeLists.txt dosyasıyla dördünü de tek komutla derleyeceksiniz.

i Neden? Hafta 1’de gördüğünüz gibi, proje dosya sayısı arttıkça elle g++ komutlarını yönetmek zorlaşır. CMake’e bir kez hangi .cpp dosyalarının hangi çalıştırılabilir dosyada birleşeceğini söylersiniz, gerisini o yönetir — bu haftaki dört ayrı hedef için de aynı iskeleti kullanacaksınız.

```
1 cmake_minimum_required(VERSION 3.20)
2 project(hafta02_temel CXX)
3 set(CMAKE_CXX_STANDARD 20)
```

Adım 12 (devamı)

```

4 | set(CMAKE_CXX_STANDARD_REQUIRED ON)
5 | add_executable(fonksiyon_demo fonksiyon_ornekleri.cpp)
6 | add_executable(parametre_demo parametre_gecme_ornekleri.cpp nokta.cpp)
7 | add_executable(bellek_demo bellek_modeli_ornekleri.cpp nokta.cpp)
8 | add_executable(int_tampon_demo int_tampon_demo.cpp int_tampon.cpp)

```

Her `add_executable` satırı, bir hedefin hangi `.cpp` dosyalarından derleneceğini tanımlar; `parametre_demo` ve `bellek_demo`'nun ikiyeşer dosyadan (kendi kaynağı + `nokta.cpp`) oluştuğuna dikkat edin.

>_ Terminal

```

$ cmake -B build
$ cmake --build build
$ ./build/fonksiyon_demo
$ ./build/parametre_demo
$ ./build/bellek_demo
$ ./build/int_tampon_demo

```

Beklenen sonuç: İlk komut `build/` klasörünü oluşturup derleme sistemini hazırlar, ikinci komut dört hedefi de tek seferde derler; sonraki dört komut her programı ayrı ayrı çalıştırır ve bu föyde daha önce gördüğünüz çıktıların birebir aynısını üretir.

Beklenen Çıktı

```

[100%] Built target fonksiyon_demo
[100%] Built target parametre_demo
[100%] Built target bellek_demo
[100%] Built target int_tampon_demo

```

Görevinizi tamamladıktan sonra, aşağıdakilerden **birini** seçip kendiniz ekleyin:

- `fonksiyon_ornekleri.cpp` içine, iki tamsayı arasındaki en büyük ortak bölüneni hesaplayan aşırı yüklenmiş bir `ebob(int a, int b)` fonksiyonu ile, adımlarını ekrana yazdırıp yazdırmayacağını belirten `bool detayli = false` varsayılan parametrelili bir `ebob(int a, int b, bool detayli)` fonksiyonu ekleyin.
- `araclar::IntTampon`'a, dizinin `i`. elemanını sınır kontrolüyle (`i`, `boyut`'un dışındaysa ekrana bir hata mesajı yazıp 0 döndüren) okuyan bir `int elemanOku(int i) const` üye fonksiyonu ekleyin; `int_tampon_demo.cpp` içindeki RAII bloğunda çağırıp sonucu yazdırın.

İpucu

Bu haftanın en sık karşılaşılan hataları: (1) aynı parametre listesiyle, yalnızca dönüş tipi farklı iki `alan` sürümü tanımlamaya çalışıp "yalnızca dönüş tipiyle ayırt edilemez" derleme hatası almak; (2) referans parametresinde `&` işaretini atlayıp fonksiyonun çağırının nesnesini değiştirmemesine şaşırarak; (3) işaretçi parametrelili bir fonksiyonun başına `nullptr` kontrolü koymayı unutmak; (4) bir `new` çağırısının karşılığı olan `delete`'i (ya da dizi için `delete[]`'i) yazmayı unutup sessizce bir bellek sızıntısı oluşturmak; (5) `IntTampon` nesnesini yanlış kapsamda (istenenden daha geniş bir blokta) oluşturup yıkıcının ne zaman çalıştığını karıştırmak.

✂ Sorun Giderme

Belirti	Olası neden	Çözüm
Aynı isimli iki fonksiyon "yeniden tanımlandı" hatası veriyor	İki fonksiyonun parametre listesi (sayı/tip) birbirinin aynısı, yalnızca dönüş tipleri farklı	Parametre listesini gerçekten farklılaştırın (ör. <code>int</code> yerine <code>double</code> parametre) ya da fonksiyona farklı bir ad verin
oteleReferans çağrıldıktan sonra nokta değişmiyor	Parametre Nokta olarak (değere göre) alınmış, & unutulmuş	oteleReferans(Nokta& n) biçiminde referans parametresi kullanın
Program çalışırken aniden çöküyor (segmentation fault)	Bir işaretçi <code>nullptr</code> kontrolü yapılmadan dereference edilmiş	İşaretçi kullanmadan önce daima <code>if (p == nullptr)</code> kontrolü ekleyin
Program uzun süre çalıştırıldığında bellek kullanımı sürekli artıyor	Bir döngüde her <code>new</code> çağrısının karşılığı olan <code>delete</code> ek-sik	Her <code>new</code> için tam olarak bir <code>delete</code> (veya <code>new[]</code> için <code>delete[]</code>) çağrıldığından emin olun
"tampon temizlendi" mesajı hiç görünmüyor	<code>IntTampon</code> nesnesi <code>main</code> 'in dışında, blok kapanmadan oluşturulmuş	Nesneyi temizlenmesini gözlemlemek istediğiniz <code>{ }</code> bloğunun içine taşıyın

5 • İzleme ➞ Aktarım — Başka Nerede?

Bugün elinizle kurduğunuz beceriler tek bir örneğe özgü değildir; dönem boyunca ve gerçek projelerde tekrar tekrar karşınıza çıkar:

- **Kitaplık projesine:** Bir `Kitap`'ı ISBN'e (`bul(std::string isbn)`) veya yayın yılına (`bul(int yıl)`) göre arayan iki işlevi aynı anlamlı `bul` adı altında toplamak, bu haftaki `alan` aşırı yüklemesinin birebir uygulamasıdır. Bir `Kitap`'ı listeye eklerken gereksiz kopyalamamak için `const Kitap&` ile geçmek, bir kaydı yerinde güncellemek için `Kitap&` ile geçmek ise bu haftaki dört-biçim karar tablosunun doğrudan tekrarıdır.
- **Bellek sorumluluğuna:** "Her `new` bir `delete` ister; otomatik olan yığın, sizin olan öbek" ayrımı dinamik bellek kullanan her yapıda geçerlidir. İlerideki Kitaplık projesinde çalışma anında büyüyen bir kitap listesi bu belleği öbekten ister; onu geri vermeyi ihmal etmek tam da bu haftanın sızıntısıdır.
- **Kaynak yönetimine (RAII):** Bugün `IntTampon` ile kurduğunuz "kaynağı nesnenin ömrüne bağlama" fikri yalnız belleğe özgü değildir; açık bir dosya, bir ağ bağlantısı ya da bir kilit gibi alınıp geri verilmesi gereken her kaynak aynı desenle yönetilir. Bu elle sarmalanmış örnek, Hafta 3'te kurucu/yıkıcının biçimsel kurallarıyla, Hafta 4'te ise `unique_ptr/shared_ptr` akıllı işaretçileriyle olgunlaşacak fikrin ilk somut adımıdır.